



Informe técnico

Sistema de recomendación personalizado para Instacart

Basket Analytics · Proyecto Final Data Science · Grupo 1

Este informe reúne el trabajo completo del proyecto. Los números de modelado, de novedad y de despliegue están confirmados por el equipo y por el repositorio. Las visualizaciones del análisis exploratorio se encuentran en el dashboard interactivo y en los notebooks del repositorio.

1. Resumen ejecutivo

Basket Analytics desarrolló un sistema de recomendación personalizado para Instacart, un supermercado online cuyo ingreso depende del tamaño del carrito y de la frecuencia de recompra. El sistema predice el próximo carrito de cada cliente a partir de su historial de compras y se entrega en una demo funcional. Sobre 26.243 usuarios de validación, el modelo elegido (LightGBM) alcanza un Hit Rate de 87,2% (frente al 46,1% del baseline de popularidad) y una mejora relativa del Recall@10 de +407% sobre ese baseline. El sistema es fuerte en recompra; el descubrimiento de productos nuevos se entrega en un bloque de sugerencias aparte, que suma 1.964 aciertos de descubrimiento sin resignar ninguno de recompra, y se mide por separado con honestidad. El impacto de negocio se plantea como hipótesis: la recomendación es llevar el sistema a una prueba en producción y validar, con KPIs de producto y un test A/B, si aumenta la incorporación de productos y el tamaño del carrito, dado que el dataset no contiene precios, costos ni comportamiento real frente a recomendaciones.

2. Contexto y problema de negocio

Instacart es un supermercado online que opera como marketplace: conecta a los clientes con shoppers que compran en tiendas físicas y entregan a domicilio. Su ingreso proviene de la comisión por orden, la suscripción y la publicidad, por lo que depende de dos variables críticas: el tamaño del carrito y la frecuencia de recompra. Como el costo de preparar y entregar una orden es en gran parte fijo, la hipótesis de negocio es que cada ítem adicional del carrito aporta ingreso con un costo incremental bajo; es una hipótesis a validar, ya que el dataset no incluye precios ni costos.

El problema es que hoy el cliente reconstruye su carrito manualmente en cada compra, repitiendo decenas de productos sin asistencia de la plataforma. Esto alarga el proceso de compra, reduce el tamaño promedio de la orden y provoca el olvido de productos habituales. Un recomendador que anticipe la compra habitual reduce esa fricción y aumenta el tamaño del carrito.

3. Objetivo y alcance

El objetivo es desarrollar un sistema personalizado que, a partir del historial de cada cliente, prediga los productos de su próximo pedido, superando un baseline de popularidad evaluado sobre la última orden real, y que se entregue en una demo funcional. El alcance central es la recompra: predecir lo que el cliente ya compra. El descubrimiento de productos nuevos se trata como una línea separada, con su propia métrica, y no se mezcla con la recomendación de recompra. Los KPIs de negocio asociados son la tasa de incorporación de productos recomendados al carrito y la cantidad promedio de ítems por pedido; su medición directa requiere clientes reales y queda para producción.

4. Datos y fuentes

Se utiliza el conjunto de datos público Instacart Market Basket Analysis (Kaggle), un conjunto relacional de seis tablas: orders, order_products__prior, order_products__train, products, aisles y departments. El volumen es de 206.209 clientes, 49.688 productos, 3,4 millones de pedidos y 32,4 millones de interacciones usuario-producto. La unidad de análisis es la interacción usuario × producto. Se conserva la granularidad de las tablas originales (no se construye una tabla monolítica). El conjunto no incluye precios ni datos demográficos, y el historial está truncado en 100 órdenes por usuario. El pedido prior representa el historial disponible y el pedido train se reserva como objetivo de evaluación. El proyecto distingue

además dos universos que no se mezclan: 131.209 usuarios evaluables (para el análisis global de baseline, candidatos y novedad) y 26.243 usuarios de validación (para la comparación oficial de los modelos).

5. Análisis exploratorio (EDA)

El EDA arrojó cuatro hallazgos que guiaron el modelado. Primero, la recompra crece con la antigüedad del cliente: sube de 24% en los nuevos a 62% en los heavy, por lo que el comportamiento se vuelve más predecible con el historial y conviene segmentar. Segundo, existe un fuerte sesgo de popularidad: cerca del 50% de las compras se explica con apenas el 1,6% del catálogo, y la cabeza del catálogo es común a casi todos los clientes, de modo que el valor está en personalizar según el historial y no en recomendar lo popular. Tercero, hay una cola larga de miles de productos que se compran rara vez. Cuarto, los clientes nuevos presentan un problema de cold-start: se sabe poco de ellos y su próxima compra suele traer productos que nunca compraron.

Los clientes se dividen en tres segmentos por profundidad de historial: Nuevos (29%, 59.741 clientes, recompra 24%), Medios (39%, 81.172 clientes, recompra 42%) y Heavy (32%, 65.296 clientes, recompra 62%). El análisis de señal mostró que la recencia, el ratio y la frecuencia predicen la recompra, mientras que la posición en el carrito y la hora de compra no aportan.

Las visualizaciones de estos hallazgos (recompra por segmento, concentración de popularidad y señal por variable) están disponibles en el dashboard interactivo y en los notebooks del repositorio (01_EDA_decisiones y 04_eda_features_para_modelo).

6. Preparación de datos e ingeniería de características

La preparación integra las seis tablas conservando su granularidad y verifica la integridad del dataset: cero productos huérfanos y cero pares (orden, producto) duplicados. Los valores nulos en “días desde la orden anterior” corresponden a la primera orden de cada usuario y se marcan con un indicador en lugar de imputarse; el valor 30 se trata como censurado (“30 días o más”) y no como un valor exacto. Los carritos muy grandes se conservan por ser reales.

Las variables se construyen a nivel usuario × producto. Las de mayor peso son la recencia (hace cuánto compró el producto), el ratio (en qué proporción de sus pedidos aparece) y la cadencia o frecuencia de reposición; se suman la recomprabilidad del producto, el vencimiento y otras señales temporales. En conjunto, las tres primeras variables explican cerca del 68% de la decisión del modelo, mientras que la popularidad global del producto pesa menos del 1%.

El pipeline está implementado en la carpeta src (submódulos data, features, models y evaluation) y se ejecuta de forma reproducible: la comparación final se regenera con un único comando (python src/models/comparar_todos.py). La lista completa de 18 variables con sus pesos está en el dashboard y en la carpeta reports.

7. Estrategia de evaluación

El protocolo de evaluación usa una separación temporal: el historial (prior) predice la última orden real de cada cliente (train), nunca a la inversa, y ninguna variable utiliza información del pedido objetivo, lo que evita la fuga de información. Los cinco sistemas se miden sobre los mismos 26.243 usuarios de validación, con el mismo código, generando un Top-10 por cliente. Las métricas son el Hit Rate como principal (a cuántos clientes se les acierta al menos un producto), acompañado por Recall@10 y Precision@10, el lift sobre el baseline, la cobertura de catálogo y una métrica de novedad para el descubrimiento. El Recall se reporta en dos formas: macro (promedio del recall por usuario, donde todos pesan igual) y micro (aciertos totales sobre productos totales, donde quien compra más pesa más); en el baseline de recompra valen 0,3271 y 0,2599. La comparación oficial se hace sobre los 26.243 usuarios de validación, mientras que el análisis de novedad y candidatos se hace sobre los 131.209 evaluables.

Durante esta etapa se detectó y corrigió una fuga en el código de los baselines: al empatar productos en las señales principales, el desempate seguía el orden del archivo, que no era neutral y favorecía a los productos efectivamente comprados. El baseline se estaba atribuyendo cerca de **cerca de 1,3 puntos** de Recall que no había ganado.

8. Modelado y comparación

El problema de “elegir diez productos” se reformuló como una pregunta binaria repetida unos 8,5 millones de veces (una por cada par cliente-producto): ¿estará este producto en su próximo pedido? El modelo devuelve una probabilidad, se ordena y se toman las diez primeras. Se construyeron cinco sistemas de complejidad creciente, medidos con el mismo protocolo.

Sistema	Hit Rate	Recall@10	Mejora Recall@10
Popularidad (baseline)	46,1%	0,070	n/a
Recompra personal	85,5%	0,327	+366%
Heurística	85,5%	0,339	+382%
Regresión logística	86,8%	0,348	+396%
LightGBM (elegido)	87,2%	0,356	+407%

La última columna es la mejora relativa del Recall@10 frente a la popularidad, no del Hit Rate; en razones equivale a 4,66×, 4,82×, 4,96× y 5,07×.

Cada escalón de complejidad gana algo y la progresión es limpia, lo que en sí es un resultado de validez. La mejora total se descompone en tres capas: cerca del 40% proviene de combinar señales a mano (heurística), un 33% de aprender los pesos (regresión logística) y un 27% de la no linealidad e interacciones (LightGBM). Para interpretar los números se calculó un techo alcanzable: un “oráculo” que viera de antemano la compra acertaría un 55%, no un 100%, porque el 40% de lo que la gente compra nunca lo había comprado (inalcanzable desde el historial) y diez lugares no alcanzan a quien compra veinticinco productos.

Nota sobre la mejora: se calcula sobre el Recall@10, no sobre el Hit Rate. Un +407% significa que el Recall del modelo es unas 5,07 veces el de la popularidad ($4,07 + 1$); no debe leerse contra el Hit Rate de 46,1%, que es otra métrica.

Además, se probaron cinco caminos para exprimir el modelo (seis variables nuevas, un ranking tipo LambdaRank, cuadruplicar los datos, la optimización de hiperparámetros con Optuna y el tamaño del carrito) y ninguno movió la aguja: el Hit Rate pasó de 87,2% a 87,22% y el Recall de 0,355 a 0,356. Es un resultado en sí mismo: el límite no es el modelo, es el dato. Falta información que el historial no tiene, como precio y promociones, y el dataset de Instacart no la trae.

9. Resultados

El sistema final entrega dos bloques separados en la pantalla, que no se mezclan. El bloque de recompra (el carrito habitual, hasta 10 productos) le acierta al menos un producto a casi 9 de cada 10 clientes (87 de cada 100), con 79.401 aciertos sobre los 26.243 usuarios de validación. El desempeño sube con la antigüedad del cliente: 83% de Hit Rate en los nuevos, 87,2% en los medios y 91,1% en los heavy.

El bloque de sugerencias (productos nuevos, del tipo “también podrías necesitar”) se entrega aparte y no le quita ningún lugar a la recompra. Su tamaño es variable por segmento (hasta 5 sugerencias para los nuevos, 2 para los medios y 1 para los heavy), definido con un criterio de producto: agregar lugares mientras el último todavía aporte relevancia (un piso del 2,5%). Suma 1.964 aciertos de descubrimiento (1.247 en nuevos, 573 en medios y 144 en heavy), sin resignar ninguno de recompra. El descubrimiento rinde al revés que la recompra: es más alto en los nuevos (Hit Rate de novedad 16,5%) y más bajo en los heavy (2,3%), lo que confirma que la novedad pesa justo donde el historial es más pobre.

Los tamaños de ambos bloques son máximos, no fijos: el carrito de recompra puede traer menos de 10 productos cuando el cliente tiene poco historial (le ocurre al 6% de los clientes y al 14% de los nuevos), y las sugerencias también pueden venir incompletas. En pantalla se comunican como hasta 10 y hasta 5 / 2 / 1. Adicionalmente, se identificaron reglas de asociación (productos que se compran juntos) medidas por lift, como base del motor de complementos.

El descubrimiento se mide con su propia lente y solo sobre los objetivos que el cliente nunca compró (555.793 en total): el Hit Rate de novedad del generador actual es de 14,24% y el Recall de novedad de 3,25%. La cobertura de catálogo se mantiene como indicador de amplitud (cerca del 44,9% en el conjunto de validación y del 71,9% sobre los evaluables) y se aclara de forma explícita que no equivale a descubrimiento: un sistema de recompra puede cubrir mucho catálogo sin recomendarle nunca a una persona algo fuera de su historial.

10. Justificación del modelo elegido

Se eligió LightGBM porque es el mejor en todas las métricas y, sobre todo, porque hay evidencia de cuánto aporta cada capa de complejidad: la no linealidad y las interacciones suman de forma medible sobre la regresión logística. No se eligió por intuición sino por la descomposición del aporte. El límite se declara con honestidad: la mejora sobre el baseline representa alrededor del 13% del camino hacia el techo teórico de Recall@10 (0,55), que es un oráculo y no una meta alcanzable; la comparación que vale es contra el baseline, donde la mejora relativa del Recall@10 va de +366% a +407%. Ese margen no está repartido igual entre segmentos: se captura el 57,8% del techo en los heavy, el 64,8% en los medios y el 73,6% en los nuevos, así que el mayor espacio para mejorar el ranking está en los clientes frecuentes.

11. KPIs de negocio e impacto

Los KPIs de negocio son la tasa de incorporación de productos recomendados al carrito (KPI principal) y la cantidad promedio de ítems por pedido (secundario). Ambos requieren clientes en vivo y un test A/B, por lo que en esta etapa no llevan un valor medido: su proxy en la evaluación offline es el Hit Rate, que anticipa la adopción. El impacto se plantea como hipótesis: si aumentan la incorporación de productos recomendados y el tamaño del carrito, debería crecer el ingreso con un costo incremental bajo. Como el dataset no contiene precios, costos ni el comportamiento real de los clientes frente a las recomendaciones, ese impacto deberá validarse en producción con KPIs de producto y una prueba A/B.

12. Despliegue y reproducibilidad

El proyecto se desarrolla en un repositorio único en GitHub, con estructura modular (data, docs, notebooks, reports, src con submódulos data, features, models y evaluation, y tests) sobre un stack de Python 3.12, DuckDB y LightGBM. El flujo de trabajo usa ramas por frente, Pull Requests con revisión entre pares y protección de la rama principal, y una suite de 59 pruebas automatizadas de las métricas y del recomendador. El pipeline es reproducible y sin rutas personales: se clona el repositorio, se crea el entorno y se instalan las dependencias, y las pruebas se corren con pytest. Una vez preparados los datos y construidas las features (según el procedimiento de SETUP.md), la comparación final se regenera con un único comando (python src/models/comparar_todos.py), de modo que dos corridas den los mismos números. La preparación parte de los seis CSV originales de Instacart, que no se versionan por su volumen y deben ubicarse en data/raw/. Las recomendaciones del modelo final para los 26.243 usuarios de validación quedaron materializadas en un archivo parquet validado con DuckDB. La demo interactiva está desplegada en Streamlit Community Cloud, en <https://basket-analytics-instacart.streamlit.app/>, y reproduce el sistema de dos bloques con un explorador de clientes reales sobre los 26.243 usuarios de validación; la columna de acierto que muestra la demo es posible porque reproduce una evaluación histórica, en la que se conoce la última orden real de cada cliente.

La integración continua está automatizada con GitHub Actions: un workflow (.github/workflows/ci.yml) se ejecuta en cada push y Pull Request a la rama principal, corre las pruebas con pytest y verifica que estén presentes los archivos que necesita el dashboard (el código, el parquet de recomendaciones y las métricas). No se usa MLflow: el seguimiento de experimentos se hace por commits y por los reportes versionados en la carpeta reports.

13. Limitaciones

El sistema está diseñado para la recompra: por construcción, el Top-10 nunca recomienda un producto que el cliente no haya comprado, de modo que la novedad del recomendador base es cero y el descubrimiento se resuelve solo con el módulo separado, hoy concentrado en unos 531 productos. La cobertura de catálogo baja con los modelos, porque se concentran en lo más recomprable; esto es coherente con el objetivo, no un defecto, pero limita la variedad. El dataset no incluye precios ni datos demográficos, y el historial está truncado en 100 órdenes por usuario, lo que acota el análisis a la dimensión de comportamiento. Finalmente, el techo alcanzable de Recall@10 (0,55) marca que una parte importante de cada pedido (los productos genuinamente nuevos) es inalcanzable desde el historial. La evaluación es offline: utiliza la última orden conocida como objetivo y no mide la adopción real, que requeriría clientes en vivo.

14. Conclusiones y próximos pasos

El equipo construyó, de forma reproducible y con una evaluación válida y sin fuga, un sistema de dos bloques (recompra más sugerencias de novedad) que supera con claridad al baseline de popularidad. Un resultado importante es que cinco

experimentos de mejora no movieron la aguja: el límite es el dato, no el modelo. La recomendación es llevar el sistema a una prueba en producción y validar su impacto con KPIs de producto y un test A/B, en lugar de darlo por probado, dado que el dataset no contiene precios ni comportamiento real. De cara al futuro, las mejoras principales pasan por enriquecer el dato con precio y promociones (lo que permitiría crecer en descubrimiento), ampliar el motor de complementos con reglas de asociación y diversificar el generador de novedad.

15. Anexos

Repositorio del proyecto: <https://github.com/Diiimas/instacart-recommender>

Demo desplegada (Streamlit): <https://basket-analytics-instacart.streamlit.app/>

Documentación en el repositorio: contrato de datos (docs/data_contract.md), diccionario de tablas procesadas (docs/data_dictionary.md), análisis de resultados y métricas (docs/analisis_resultados_y_metricas.md) y configuración del entorno (SETUP.md). Las métricas completas y reproducibles están en reports/comparacion_final.csv. Notebooks: 01_EDA_decisiones, 02_calidad_data, 03_baseline y 04_eda_features_para_modelo.

La demo publicada incluye el dashboard interactivo con todas las visualizaciones del análisis; por eso no se adjuntan figuras por separado.